

Reporte Técnico de Actividades Práctico- Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	- Soledad Buri - Gyna Yupanqui
Asignatura	Estructura de datos
Ciclo	3 A
Unidad	2
Resultado de aprendizaje de la unidad	Aplica los métodos de ordenación y búsqueda en la resolución de problemas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Título de la Práctica	Ordenación básica en Java: Burbuja, Selección e Inserción
Nombre del Docente	Andrés Roberto Navas Castellanos
Fecha	Jueves 13 de noviembre Viernes 14 de noviembre
Horario	07h30 – 10h30 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	5 horas

2. Objetivo(s) de la Práctica

- Implementar y comparar tres algoritmos de ordenación in-place sobre arreglos pequeños, y validar su funcionamiento con trazas y casos de prueba reproducibles.

3. Materiales, Reactivos, Equipos y Herramientas

- Guía de pruebas con datasets y salidas esperadas.

4. Equipos y Herramientas

- JDK OpenJDK (obligatorio).
- IDE: Visual Studio Code (extensión “Extension Pack for Java”) o IntelliJ IDEA Community.
- Sistema de control de versiones: Git; repositorio en GitHub.
- EVA/Moodle institucional: para entrega de evidencias.
- Herramientas de documentación: README Markdown, editor ofimático (Google Docs/LibreOffice/Word).

5. Procedimiento / Metodología Ejecutada

Enfoque metodológico: ABPr (Aprendizaje Basado en Proyectos).

Inicio

- Presentación breve de objetivos y alcance (qué implementar y cómo verificar).
- Organización individual o en parejas; crear repo Git y rama de trabajo.
- Revisión de firmas y criterios de evaluación (rúbrica).
- Lineamientos de uso responsable de IA.

Desarrollo

- Implementar en este orden: Inserción, Selección, Burbuja (facilita comparar).

Estructura del proyecto

- Paquete `ed.u2.sorting`.
- Clases sugeridas: `InsertionSort`, `SelectionSort`, `BubbleSort`, `SortingDemo`, `SortingUtils`.

Firmas estándar (Java)

```
public final class InsertionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
  
public final class SelectionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
  
public final class BubbleSort {  
    public static void sort(int[] a) { /* ... */ }  
}
```

- Todas **in-place** (modifican el arreglo recibido).

- Agregar sobrecarga opcional `sort(int[] a, boolean trace)` para imprimir trazas.

Inserción (recomendado empezar aquí)

- Bucle externo $i=1..n-1$; insertar $a[i]$ en $0..i-1$.
- **Trazas:** imprimir posiciones movidas y arreglo resultante por iteración.

- **Selección**

- Para cada i , hallar **mínimo** en $i..n-1$ y cambiar por $a[i]$.
- Contabilizar **swaps** (pocos intercambios).

- **Burbuja**

- Pasadas sucesivas; intercambiar adyacentes “mal ordenados”.
- Optimización: corte temprano si no hubo **swaps** en una pasada.

- **Casos bordes**

- Arreglo vacío, tamaño 1, todos iguales, ya ordenado, orden inverso, con duplicados.

- **Trazas y validación**

- Ejecutar `SortingDemo` con datasets de la sección 10.
- Guardar capturas/salidas en TXT para subir al EVA.

Cierre

- Comparación cualitativa: cuándo usar cada algoritmo (datos pequeños, casi ordenados, número de swaps, legibilidad de traza).
- Revisión de evidencias y dudas frecuentes.
- Recordatorio de entregables y modo de verificación automática en EVA.

6. Resultados

InsertionSort

```
== PRUEBA: InsertionSort (Inserción) ==  
Original (A): [8, 3, 6, 3, 9]  
Iteración 1 (insertando 3): [3, 8, 6, 3, 9]  
Iteración 2 (insertando 6): [3, 6, 8, 3, 9]  
Iteración 3 (insertando 3): [3, 3, 6, 8, 9]  
Iteración 4 (insertando 9): [3, 3, 6, 8, 9]  
Ordenado: [3, 3, 6, 8, 9]  
Esperado: [3, 3, 6, 8, 9]
```

BubbleSort Inverso

```
== PRUEBA: BubbleSort (Burbuja) - Inverso ==  
Original (B): [5, 4, 3, 2, 1]  
Pasada 1: [4, 3, 2, 1, 5]  
Pasada 2: [3, 2, 1, 4, 5]  
Pasada 3: [2, 1, 3, 4, 5]  
Pasada 4: [1, 2, 3, 4, 5]  
Ordenado: [1, 2, 3, 4, 5]  
Esperado: [1, 2, 3, 4, 5]
```

BubbleSort

```
== PRUEBA: BubbleSort (Burbuja) ==  
Original (C): [1, 2, 3, 4, 5]  
Pasada 1: [1, 2, 3, 4, 5]  
Corte temprano en pasada 1: Arreglo ya ordenado.  
Ordenado: [1, 2, 3, 4, 5]  
Esperado: [1, 2, 3, 4, 5]
```

SelectionSort con Duplicado

```
== PRUEBA: SelectionSort con Duplicados ==  
Original (D): [2, 2, 2, 2]  
Iteración 0 (mínimo 2 puesto en pos 0): [2, 2, 2, 2]  
Iteración 1 (mínimo 2 puesto en pos 1): [2, 2, 2, 2]  
Iteración 2 (mínimo 2 puesto en pos 2): [2, 2, 2, 2]  
Total de intercambios (swaps) realizados: 0  
Ordenado: [2, 2, 2, 2]  
Esperado: [2, 2, 2, 2]
```

SelectionSort

```
== PRUEBA: SelectionSort (Selección) ==  
Original (E): [9, 1, 8, 2]  
Iteración 0 (mínimo 1 puesto en pos 0): [1, 9, 8, 2]  
Iteración 1 (mínimo 2 puesto en pos 1): [1, 2, 8, 9]  
Iteración 2 (mínimo 8 puesto en pos 2): [1, 2, 8, 9]  
Total de intercambios (swaps) realizados: 2  
Ordenado: [1, 2, 8, 9]  
Esperado: [1, 2, 8, 9]
```

ENLACE AL GIT:

https://github.com/Zoe777zz/TALLER_05

Tabla Ejecutada

Algoritmo	Arreglo de Prueba	Resultado de la Traza
InsertionSort	A = [8, 3, 6, 3, 9]	Realizó 4 iteraciones (i=1 a i=4) para ordenar el arreglo
BubbleSort	B = [5, 4, 3, 2, 1]	El arreglo inverso forzó a hacer las 4 pasadas completas
BubbleSort	C = [1, 2, 3, 4, 5]	Hizo solo 1 pasada y se detuvo gracias al corte temprano
SelectionSort	D = [2, 2, 2, 2]	Arreglo de duplicados se modificó y reportó 0 swaps
SelectionSort	E = [9, 1, 8, 2]	Hizo 3 iteraciones y reportó un total de 2 swaps

7. Preguntas de Control

- **¿Por qué Inserción es preferible con datos casi ordenados?**

Es preferible por el tiempo de ejecución ya que cada paso que da el algoritmo se toma un elemento y se inserta en su posición correcta dentro de la parte ya ordenada entonces si el arreglo ya esta muy cerca de la posición final significa que se realizara muy pocas comparaciones y desplazamientos.

- **¿Qué propiedad hace que Selección use pocos swaps? ¿Qué compromisos tiene?**

Solo realiza un swap por pasada del bucle externo, en cada pasada, el algoritmo busca el elemento más pequeño o más grande en la parte no ordenada del arreglo, al finalizar la búsqueda lo intercambia con el elemento en la posición actual del límite.

- **¿Cómo implementarías el corte temprano en Burbuja y qué caso reduce significativamente?**

Este corte temprano se implementaría agregando una bandera el cual verifique si durante una pasada completa del arreglo se realizo al menos un intercambio llegando a reducir significativamente en el mejor caso cuando el arreglo ya esta ordenado.

- **¿Cuál(es) de los tres puede(n) ser estable y en qué condiciones?**

La inserción y burbuja son los más estables, ya que la inserción desplaza un elemento mayor o igual al que se etapa insertando y la burbuja solo se intercambia si el primero es estrictamente mayor que el segundo.

- **Menciona dos casos bordes que deben probarse siempre.**

En casos con arreglo ya ordenado, y en otro caso con un arreglo ordenado, pero en sentido inverso.

Soledad

- Ordenar números positivos y negativos en un mismo arreglo

Gyna

- Que en un arreglo exista un número pequeño, uno grande, es decir intercalado

8. Conclusiones

- Al poner en práctica los tres tipos de algoritmos de ordenación se pudo apreciar sus diferencias y su funcionamiento con respecto a su eficiencia el momento de trabajar en pequeños arreglos.
- Se observó que todos los métodos empleados cumplen con su manera de ordenar, pero si presentan una variación en el número de operaciones realizadas.

9. Recomendaciones

- Realizar pruebas con arreglos con una dimensión un poco más grandes para tener una visualización mejor de los arreglos.
- Para una mejor comprensión del uso de los algoritmos ocupados se puede realizar arreglos a menor trazándolos y viendo cada paso de manera más comprensible para poner en práctica con código.

10. Bibliografía / Referencias

- [1] “Insertion sort (with code in Python/C++/Java/C)”, *Programiz.com*. [En línea]. Disponible en: <https://www.programiz.com/dsa/insertion-sort>. [Consultado: 15-nov-2025].
- [2] “Bubble sort”, *Programiz.com*. [En línea]. Disponible en: <https://www.programiz.com/dsa/bubble-sort>. [Consultado: 15-nov-2025].
- [3] “Selection sort (with code in Python/C++/Java/C)”, *Programiz.com*. [En línea]. Disponible en: <https://www.programiz.com/dsa/selection-sort>. [Consultado: 16-nov-2025].